# Embedded Systems Project Showcase

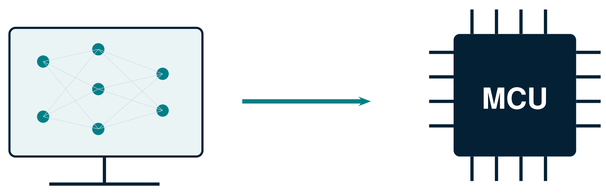
## Integrated Computer Engineering (Batch 23)

# Automated Sim-to-Real Deployment Framework for Reinforcement Learning Control on Resource Constrained Microcontrollers

Nidharshan V. Yasantha Niroshana Odil Janandith Sulochana Sooriyaarachchi

## 1. PROBLEM SCENARIO

**Learn in simulation. Run on a robot.**




**TRAINING COMPUTER**

Train in simulation
Save the learned policy

**REAL ROBOT**

ESP32 runs the controller
Small onboard memory

### Deployment barriers

1. Rewrite the learned controller as code
2. Fit the code into a small controller
3. Connect sensor readings to motors

### The missing link

A robot learns in a computer simulation. Its **policy** chooses movements from sensor readings. Moving this learned controller to a real robot usually requires manual coding and hardware setup.

**Example task: self-rising**

A fallen robot must stand up using a tilt sensor and two servo motors.

## 2. FROM A VIRTUAL ROBOT TO A REAL ROBOT

Training happens on the computer; the generated code runs on the robot.

**SIMULATE** — Build a virtual robot — MuJoCo

→ **TRAIN** — Learn by trial and error — PPO training

→ **CONVERT** — Generate robot code — simtoreal library

→ **RUN** — Control the real robot — ESP32

Robot settings: sensors, motors and pins

**GENERATED FILES → COPY TO THE ESP32**

Policy code + `main.py` + sensor and motor drivers

**Read sensors** → **Choose a movement** → **Move motors**

## 3. HOW THE LIBRARY WORKS

**Start conversion on your computer**

`simtoreal model.zip -c config.yaml` (command-line entry point)

**Coordinate conversion**
`converter.py`
Check input/output counts
Choose the export format

→ **Read learned decisions**
`loaders.py`: actor weights
→ **Write policy code**
`exporter.py`: .h / .py / .mpy

→ **Read robot settings**
Sensor inputs + motor outputs
`config.yaml`: classes + pins
→ **Connect hardware + policy**
`interface.py` → `main.py`

`main.py` imports driver classes, sets pins and joins the control loop.

**ON ESP32** Read sensors → Policy decides → Move motors → Repeat

## 4. CORE FEATURES

### 1. Keep the learned controller

Trained PPO model → Decision network

Training-only components are left out.

### 2. Choose a robot-ready format

| `.h` | `.py` | `.mpy` |
|---|---|---|
| C header | Readable code | Compiled code |

Use .h in C firmware; .py / .mpy in MicroPython.

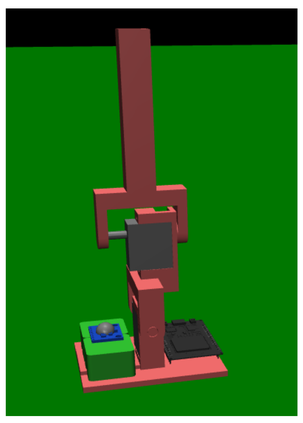
### 3. Connect sensors and motors

Robot settings → `main.py`

Read sensors, run the policy, move motors

## 5. EXAMPLE: A ROBOT THAT STANDS UP

**COMPUTER SIMULATION**



Learn how to stand up in MuJoCo

**PHYSICAL ROBOT**



ESP32 + tilt sensor + two servo motors

**5 sensor/state inputs** → **Learned decisions** → **2 motor commands**

## 6. LITERATURE REVIEW

| RESEARCH | WHAT IT IS ABOUT | HOW USED HERE |
|---|---|---|
| **Proximal Policy Optimization Algorithms** Schulman et al., 2017 | Learning useful movements through trial and error | Trains the robot controller; we export its learned decisions |
| **Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World** Tobin et al., IROS 2017 | Varying simulated scenes to improve real-world transfer | Vary robot physics and sensor noise during training |
| **TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems** David et al., MLSys 2021 | Running learned models on limited-memory controllers | Design reference for the robot's small memory budget |
| **RLtools: A Fast, Portable Deep Reinforcement Learning Library for Continuous Control** Eschmann, Albani & Loianno, JMLR 2024 | Fast C++ learning and decisions across different devices | Reference for running learned controllers on small devices |

## 7. RESULTS

| CHECK | RESULT |
|---|---|
| Export: maximum error | $< 3 \times 10^{-6}$ |
| Simulated get-up success | **97.2%** |
| Generated ESP32 loop | **100/100 cycles** |
| MicroPython decision time | **155 ms** |
| MicroPython policy RAM | **119.6 kB** |

**CONCLUSION**

Accurate conversion; faster inference is needed for 50 Hz control.

**EXPLORE THE PROJECT**

github.com/NidharVignesh/sim2real-framework

Scan for code + examples

**Organized by**

**Department of Computer Science & Engineering**

**University of Moratuwa**